

Slide 2: Chapter Overview - Main Topics

This slide outlines 9 major topics covered in the chapter:

- **Features of Good Relational Design** - Principles for well-designed databases
- **Atomic Domains and First Normal Form** - Ensuring data atomicity
- **Decomposition Using Functional Dependencies** - Breaking down relations based on FDs
- **Functional Dependency Theory** - Mathematical foundation of FDs
- **Algorithms for Functional Dependencies** - Computational methods for FD analysis
- **Decomposition Using Multivalued Dependencies** - Advanced decomposition using MVDs
- **More Normal Forms** - Higher normalization levels (BCNF, 3NF, 4NF)
- **Database-Design Process** - Complete design methodology
- **Modeling Temporal Data** - Handling time-dependent information

Slide 3: The Banking Schema

This slide presents a comprehensive **real-world database example** using a banking system. It defines 15 relations with their attributes (underscored attributes are primary keys):

- **Entity relations:** branch, customer, loan, account, employee
- **Relationship relations:** borrower, depositor, account_branch, loan_branch, cust_banker, works_for, payment
- **Specialization relations:** savings_account, checking_account, dependent_name

This schema will be used throughout the presentation to illustrate concepts.

Slide 4: Combine Schemas? - The Problem

Key Concept: Information Redundancy

This slide shows the **danger of combining relations**. When combining `borrower` and `loan` into `bor_loan(customer_id, loan_number, amount)`:

- Multiple customers can borrow the same loan (L-100)
- The amount (10000) is **repeated** for each customer
- **Problem:** Redundant data storage wastes space and creates update anomalies

Original (Separate)	Combined (Merged)
No repetition	L-100 amount repeated 3 times
Clean separation of concerns	Data duplication

Slide 5: A Combined Schema Without Repetition

Key Concept: When Combination Works

This slide shows the **opposite scenario** - combining `loan` and `loan_branch`:

- When combined into `loan_amt_br(loan_number, amount, branch_name)`:
 - **No repetition occurs** because each loan is assigned to exactly ONE branch
 - The relationship is 1-to-many (one branch can have many loans), which doesn't cause duplication

Key Insight: Not all combinations cause problems. Combination only causes redundancy when the combined relation doesn't have proper key constraints.

Slide 6: What About Smaller Schemas? - Introduction to Functional Dependencies

Key Concept: Recognizing When to Decompose

This critical slide introduces the fundamental problem:

- If we had `bor_loan` with repetition, how would we know to split it?
- **Solution:** Use **Functional Dependencies (FDs)**

- Rule: `loan_number` $\rightarrow$ `amount` (loan number determines amount)
- Because `loan_number` is NOT a candidate key in `bor_loan`, repetition occurs
- This FD indicates we **must decompose** the relation

Warning: Not all decompositions are good. Example of bad decomposition:

- Splitting `employee` into `employee1(employee_id, employee_name)` and `employee2(employee_name, telephone_number, start_date)`
- **Problem:** We **cannot reconstruct** the original relation (lossy decomposition)

Slide 7: A Lossy Decomposition

Key Concept: Information Loss Through Decomposition

This slide visually demonstrates why the previous decomposition is **lossy**:

- **Original employee:** Two different Kims with IDs 123-45-6789 and 987-65-4321
- **After decomposition:** When we join `employee1` and `employee2` back:
 - We get 4 tuples instead of 2 (Cartesian product of names)
 - Cannot tell which Kim works with which phone number
 - **Original information is lost**

Lesson: Decomposition must be **lossless-join** - we must be able to reconstruct the original relation perfectly.

Slide 8: First Normal Form (1NF) - Part 1

First Normal Form (1NF): Eliminating Duplicate Records

A table is in 1NF if it satisfies the following conditions:

- All columns contain atomic values (i.e., indivisible values).
- Each row is unique (i.e., no duplicate rows).
- Each column has a unique name.
- The order in which data is stored does not matter.

Example of 1NF Violation: If a table has a column "Phone Numbers" that stores multiple phone numbers in a single cell, it violates 1NF. To bring it into 1NF, you need to separate phone numbers into individual rows.

Key Concept: Atomicity of Domains

- **Atomic domain:** Elements are indivisible units (cannot be broken down meaningfully)
- **Non-atomic examples:**
 - Set of names (can be split into first, middle, last)
 - Composite attributes
 - Coded IDs like CS101 (can be parsed as department + number)

Definition: A schema R is in **1NF** if all attributes have atomic domains.

Benefits:

- Simplifies storage and querying
- Prevents redundant data storage
- Encourages proper normalization

Slide 9: First Normal Form (1NF) - Part 2

Key Concept: Atomicity is Context-Dependent

- Atomicity depends on **how data is used**, not inherent properties
- **Example:** Roll numbers like "CS0012" or "EE1127"
 - Normally atomic (indivisible strings)
 - But if we extract first 2 characters to find department, they're NOT atomic
 - **Bad practice:** Encodes information in application rather than database

Principle: Keep domains atomic; extract information during query, not during storage.

Slide 10: Goal - Devise a Theory

Key Concept: The Central Problem Statement

This slide sets the overall objective:

1. **Determine if relation R is in "good" form** (well-normalized)
2. **If not, decompose into $\{R_1, R_2, \dots, R_n\}$ where:**
 - Each relation is in good form
 - Decomposition is **lossless-join**
 - Preferably **dependency preserving**

Theory basis:

- **Functional Dependencies (FDs)**
- **Multivalued Dependencies (MVDs)**

Slide 11: Functional Dependencies - Introduction

Key Concept: What Are Functional Dependencies?

- **FDs are constraints** on legal relation values
- **Definition:** $X \rightarrow Y$ means the value of X **uniquely determines** the value of Y
- **Generalization of keys:** A key is a special case of FD
- **Example:** `branch_name` $\rightarrow$ `assets` (branch name determines its assets)

Slide 12: Functional Dependencies (Continued) - Formal Definition

Key Concept: Mathematical Definition of FDs

Definition: Let $\alpha \subseteq R$ and $\beta \subseteq R$. The FD $\alpha \rightarrow \beta$ holds on R if:

- For any two tuples t_1 and t_2 in any legal relation $r(R)$
- If $t_1[\alpha] = t_2[\alpha]$ (same α values), then $t_1[\beta] = t_2[\beta]$ (same β values)
- **Formula:** $t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$

Example from slide:

A	B
1	4

1	5
3	7

- $A \rightarrow B$ does **NOT** hold ($A=1$ maps to both 4 and 5)
- $B \rightarrow A$ **DOES** hold (each B value maps to one A value)

Slide 13: Functional Dependencies - Keys and Superkeys

Key Concept: Relationship Between FDs and Keys

Definitions:

- **Superkey K:** If $K \rightarrow R$ (K determines all attributes)
- **Candidate key K:**
 - $K \rightarrow R$ (is a superkey)
 - No subset of K has this property (is minimal)

Key insight: FDs express constraints beyond just keys. Example:

- In `bor_loan(customer_id, loan_number, amount)`:
 - `loan_number → amount` is an FD
 - But this is NOT a key constraint (`loan_number` isn't a candidate key)

Slide 14: Use of Functional Dependencies

Key Concept: Two Main Uses

FDs are used to:

1. **Test relations:** Check if instance satisfies given FDs
 - If r satisfies FD set F, we say r **satisfies** F
2. **Specify constraints:** Describe valid databases
 - If F applies to all legal instances, we say F **holds on** R

Important distinction:

- A specific instance may satisfy an FD even if it doesn't hold on the schema
- Example: A specific loan instance might accidentally satisfy $\text{amount} \rightarrow \text{customer_name}$ even though it shouldn't hold

Slide 15: Trivial Functional Dependencies

Key Concept: FDs That Always Hold

Trivial FD: Satisfied by all relation instances automatically

Definition: $\alpha \rightarrow \beta$ is trivial if $\beta \subseteq \alpha$ (right side is subset of left side)

Examples:

- $\text{customer_name, loan_number} \rightarrow \text{customer_name}$ (trivial)
- $\text{customer_name} \rightarrow \text{customer_name}$ (trivial)
- $\text{loan_number} \rightarrow \text{amount, loan_number}$ (trivial)

Why care: When testing for normal forms, we ignore trivial FDs as they provide no constraints.

Slide 16: Closure of Functional Dependencies

Key Concept: Implied Dependencies

Definition: The **closure** F^+ is the set of ALL FDs logically implied by F

Example:

- Given: $A \rightarrow B$ and $B \rightarrow C$
- Can infer: $A \rightarrow C$ (by transitivity)
- Therefore $A \rightarrow C \in F^+$

Importance:

- To check if a relation satisfies all FDs, we need to consider F^+ , not just F

- Some violations might only appear in implied dependencies

Slide 17: Boyce-Codd Normal Form (BCNF) - Definition

Key Concept: Strongest Normal Form

Definition: R is in BCNF if for all non-trivial FDs $\alpha \rightarrow \beta$ in F^+ :

- Either $\alpha \rightarrow \beta$ is trivial ($\beta \subseteq \alpha$), OR
- α is a superkey for R

In plain English:

- The only FDs that should exist are those determined by candidate keys
- Non-key attributes should not determine anything

Example (violation):

- `bor_loan(customer_id, loan_number, amount)` with `loan_number → amount`
- `loan_number` is NOT a superkey
- Therefore violates BCNF

Slide 18: Decomposing a Schema into BCNF

Key Concept: How to Fix BCNF Violations

Algorithm: When non-trivial FD $\alpha \rightarrow \beta$ violates BCNF:

- Decompose into two relations:
 - a. $R_1 = \alpha \cup \beta$ (attributes causing violation plus dependent)
 - b. $R_2 = R - (\beta - \alpha)$ (remove only β attributes that aren't in α)

Example:

- `bor_loan(customer_id, loan_number, amount)` with `loan_number → amount`
- $\alpha = \text{loan_number}$, $\beta = \text{amount}$

- $R_1 = (\text{loan_number}, \text{amount})$
- $R_2 = (\text{customer_id}, \text{loan_number})$

Slide 19: BCNF and Dependency Preservation

Key Concept: The Trade-off

Problem: BCNF decomposition may not be **dependency preserving**

- Dependency preserving means: Can check all FDs by examining individual relations
- If not preserved: Must compute joins to verify FD compliance (expensive)

Trade-off:

- Sometimes cannot achieve both BCNF AND dependency preservation
- Solution: Relax to **Third Normal Form (3NF)** which can achieve both

Slide 20: Third Normal Form (3NF) - Definition

Key Concept: Weaker But More Practical Normal Form

Definition: R is in 3NF if for all non-trivial FDs $\alpha \rightarrow \beta$ in F^+ :

- $\alpha \rightarrow \beta$ is trivial ($\beta \subseteq \alpha$), OR
- α is a superkey, OR
- **Each attribute in $\beta - \alpha$ is in some candidate key**

The third condition is the key difference from BCNF - allows some "redundancy with a safety valve"

Properties:

- If R is in BCNF, it's automatically in 3NF
- 3NF is weaker (less restrictive) than BCNF
- There always exists a lossless-join, dependency-preserving decomposition into 3NF

Slide 21: Goals of Normalization - Summary

Key Concept: Three Objectives

Given relation scheme R with FD set F, the goal is:

1. **Test:** Decide if R is in "good" form
2. **Decompose:** If not, create $\{R_1, R_2, \dots, R_n\}$ where:
 - Each is in good form
 - **Lossless-join decomposition** (can reconstruct original)
 - **Preferably dependency-preserving** (can check FDs on individual relations)

This is the driving principle for all normalization work.

Slide 22: How Good is BCNF? - Part 1

Key Concept: BCNF Limitations

Problem: Some BCNF schemas still have issues!

Example: `classes(course, teacher, book)` where:

- Course can have multiple teachers
- Each course has multiple required textbooks
- Any teacher can teach any required textbook combination

FDs: None! (course doesn't uniquely determine teacher or book)

- Therefore: **Relation IS in BCNF**
- But still has problems!

Slide 23: How Good is BCNF? - Part 2

Key Concept: Insertion Anomalies in BCNF

Problem with BCNF relation:

| course | teacher | book |

database	Avi	DB Concepts
database	Avi	Ullman
database	Hank	DB Concepts
database	Hank	Ullman
database	Sudarshan	DB Concepts
database	Sudarshan	Ullman

Insertion anomaly: If new teacher Marilyn joins, must insert 2 tuples (for each textbook)

- Violates "one fact, one tuple" principle
- Creates update complexity

Slide 24: How Good is BCNF? - Part 3 (Better Decomposition)

Key Concept: Higher Normal Forms Needed

Better decomposition into two relations:

teaches:

course	teacher
database	Avi
database	Hank
operating systems	Avi

text:

course	book
database	DB Concepts
database	Ullman
operating systems	OS Concepts

Advantage:

- No insertion anomalies
- Independent relationships
- Suggests need for **Fourth Normal Form (4NF)** with multivalued dependencies

Slide 25: Functional Dependency Theory - Introduction

Key Concept: Formal Mathematical Foundation

This slide transitions to formal theory that enables:

1. Determining which FDs are logically implied by given FDs
2. Developing algorithms for decomposing into BCNF and 3NF
3. Testing if decomposition preserves dependencies

Slide 26: Closure of FDs - Computing F^+

Key Concept: Armstrong's Axioms

Three inference rules (Armstrong's Axioms) to compute F^+ :

1. **Reflexivity:** If $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ (always true)
2. **Augmentation:** If $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$ (add attributes to both sides)
3. **Transitivity:** If $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$ (chain dependencies)

Properties:

- **Sound:** Only generates true FDs
- **Complete:** Generates ALL true FDs
- Can be used to compute all members of F^+

Slide 27: Example of Computing F^+

Key Concept: Applying Armstrong's Axioms

Given:

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$

Members of F^+ that can be derived:

1. **A $\rightarrow$ H**: By transitivity ($A \rightarrow B, B \rightarrow H$)
2. **AG $\rightarrow$ I**: By augmentation ($A \rightarrow C$ gives $AG \rightarrow CG$) then transitivity ($AG \rightarrow CG, CG \rightarrow I$)
3. **CG $\rightarrow$ HI**: By combining $CG \rightarrow I$ and $CG \rightarrow H$

Each can be derived using the axioms systematically.

Slide 28: Procedure for Computing F^+

Key Concept: Algorithm

Pseudocode:

```

 $F^+ = F$ 
repeat
  for each FD  $f$  in  $F^+$  :
    apply reflexivity and augmentation on  $f$ 
    add resulting FDs to  $F^+$ 
  for each pair of FDs  $f_1, f_2$  in  $F^+$  :
    if  $f_1$  and  $f_2$  can be combined using transitivity:
      add resulting FD to  $F^+$ 
until  $F^+$  doesn't change

```

Note: Alternative (more efficient) procedure will be shown later using attribute closure.

Slide 29: Closure Rules - Additional Useful Rules

Key Concept: Derived Rules

Additional rules that can be derived from Armstrong's Axioms:

1. **Union:** If $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$, then $\alpha \rightarrow \beta\gamma$ (combine right sides)
2. **Decomposition:** If $\alpha \rightarrow \beta\gamma$, then $\alpha \rightarrow \beta$ and $\alpha \rightarrow \gamma$ (split right side)
3. **Pseudotransitivity:** If $\alpha \rightarrow \beta$ and $\gamma\beta \rightarrow \delta$, then $\alpha\gamma \rightarrow \delta$ (generalized transitivity)

These simplify manual computation of F^+ .

Slide 30: Closure of Attribute Sets

Key Concept: Computing α^+ (More Efficient)

Definition: α^+ = set of attributes functionally determined by α under F

Algorithm (more efficient than computing full F^+):

```
result =  $\alpha$ 
while (result changes):
    for each  $\beta \rightarrow \gamma$  in  $F$ :
        if  $\beta \subseteq \text{result}$ :
            result = result  $\cup \gamma$ 
```

Advantage: $O(n^2)$ instead of exponential time for computing closures.

Slide 31: Example of Attribute Closure

Key Concept: Applying the Algorithm

Given: $R=(A,B,C,G,H,I)$, $F=\{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$

Compute $(AG)^+$:

1. result = AG
2. Apply $A \rightarrow B, A \rightarrow C$: result = ABCG
3. Apply $CG \rightarrow H$ (since $CG \subseteq ABCG$): result = ABCGH
4. Apply $B \rightarrow H$: already $H \in \text{result}$
5. Apply $CG \rightarrow I$ (since $CG \subseteq ABCGH$): result = ABCGHI

Is AG a candidate key?

- Is AG a superkey? Does $(AG)^+ \supseteq R$? YES (ABCGHI $\supseteq$ ABCGHI)
- Is any subset a superkey? Check $(A)^+$ and $(G)^+$ - neither contains R
- Therefore: **AG IS a candidate key**

Slide 32: Uses of Attribute Closure

Key Concept: Three Important Applications

1. Testing for superkey:

- Compute α^+
- If $\alpha^+ \supseteq R$, then α is a superkey

2. Testing FD membership:

- To check if $\alpha \rightarrow \beta$ holds
- Compute α^+ , check if $\beta \subseteq \alpha^+$
- Simple, efficient, very useful

3. Computing F^+ closure:

- For each subset $\gamma \subseteq R$, compute γ^+
- For each $S \subseteq \gamma^+$, output $\gamma \rightarrow S$
- Generates complete F^+

Slide 33: Canonical Cover - Introduction

Key Concept: Minimal Set of FDs

Problem: FD sets may contain **redundancy**:

- Entire FDs can be redundant (e.g., $A \rightarrow C$ redundant if $A \rightarrow B$ and $B \rightarrow C$ exist)
- Parts of FDs can be redundant (on left or right sides)

Example:

- $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can simplify to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
- $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can simplify to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

Canonical cover F_c : Minimal equivalent set with:

- No redundant FDs
- No redundant attributes in any FD

Slide 34: Extraneous Attributes - Definition

Key Concept: Identifying Redundant Parts

Definitions:

- **Extraneous in α** (left side): Attribute $A \in \alpha$ is extraneous if removing it doesn't change logical implications
 - F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$
- **Extraneous in β** (right side): Attribute $A \in \beta$ is extraneous if removing it preserves logical implications
 - $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F

Examples:

- In $\{A \rightarrow C, AB \rightarrow C\}$: B is extraneous in $AB \rightarrow C$ (since $A \rightarrow C$ already covers it)
- In $\{A \rightarrow C, AB \rightarrow CD\}$: C is extraneous in $AB \rightarrow CD$ (since $AB \rightarrow C$ can be inferred)

Slide 35: Testing for Extraneous Attributes

Key Concept: Algorithms

To test if $A \in \alpha$ is extraneous (left side):

1. Compute $(\alpha - A)^+$ using all dependencies in F
2. If $(\alpha - A)^+ \supseteq \beta$, then A is extraneous

To test if $A \in \beta$ is extraneous (right side):

1. Compute α^+ using $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$
2. If $\alpha^+ \supseteq A$, then A is extraneous in β

Slide 36: Canonical Cover - Definition and Algorithm

Key Concept: Computing the Minimal Set

Definition: Canonical cover F_c is a set where:

- F logically implies all FDs in F_c
- F_c logically implies all FDs in F
- No FD in F_c contains extraneous attributes
- Each left side of FD is unique

Algorithm:

```
repeat
    Apply union rule:  $\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2$  becomes  $\alpha \rightarrow \beta_1\beta_2$ 
    Find FD  $\alpha \rightarrow \beta$  with extraneous attribute
    If found, delete the extraneous attribute
until no changes
```

Slide 37: Example of Computing Canonical Cover

Key Concept: Step-by-Step Process

Given: $R=(A,B,C)$, $F=\{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$

Step 1: Combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$

- Result: $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$

Step 2: Check if A is extraneous in $AB \rightarrow C$

- Does $B \rightarrow C$ exist? Yes!
- Therefore A is extraneous
- Result: $\{A \rightarrow BC, B \rightarrow C\}$

Step 3: Check if C is extraneous in $A \rightarrow BC$

- Is $A \rightarrow C$ implied by others?
- Yes! $A \rightarrow B$ and $B \rightarrow C$ gives $A \rightarrow C$ by transitivity

- Therefore C is extraneous in $A \rightarrow BC$
- Result: $\{A \rightarrow B, B \rightarrow C\}$

Final canonical cover: $\{A \rightarrow B, B \rightarrow C\}$

Slide 38: Lossless-Join Decomposition

Key Concept: Ensuring Reconstruction

Definition: Decomposition of R into R_1 and R_2 is lossless-join if:

- For all legal relations r on R: $r = \Pi_{\{R_1\}}(r) \bowtie \Pi_{\{R_2\}}(r)$
- Can perfectly reconstruct original relation

Theorem: Decomposition is lossless-join if and only if:

- $R_1 \cap R_2 \rightarrow R_1$ is in F^+ , OR
- $R_1 \cap R_2 \rightarrow R_2$ is in F^+

In other words, the common attributes must form a superkey for at least one of the relations.

Slide 39: Example of Lossless vs Lossy Decomposition

Key Concept: Comparing Two Decompositions

Given: $R=(A,B,C)$, $F=\{A \rightarrow B, B \rightarrow C\}$

Decomposition 1 (GOOD):

- $R_1=(A,B)$, $R_2=(B,C)$
- $R_1 \cap R_2 = \{B\}$
- $B \rightarrow BC$ (B determines all of R_2) ✓ Lossless
- Dependency preserving ✓

Decomposition 2 (PROBLEMATIC):

- $R_1=(A,B)$, $R_2=(A,C)$

- $R_1 \cap R_2 = \{A\}$
- $A \rightarrow AB$ ✓ Lossless
- NOT dependency preserving ✗ (must compute join to check $B \rightarrow C$)

Slide 40: Dependency Preservation - Definition

Key Concept: Checking FDs Locally

Definition: Let F_i = set of FDs from F^+ containing only attributes of R_i

Decomposition is **dependency preserving** if:

- $(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$

Why: If FDs can be checked on individual relations without computing joins, updates are efficient.

If not preserved: Must compute joins to verify constraints (expensive).

Slide 41: Testing Dependency Preservation Algorithm

Key Concept: Efficient Test

To check if FD $\alpha \rightarrow \beta$ is preserved:

```

result =  $\alpha$ 
while (result changes):
    for each  $R_i$  in decomposition:
         $t = (\text{result} \cap R_i)^+ \cap R_i$  (closure in  $R_i$ , intersected with  $R_i$ )
        result = result  $\cup$  t
if result contains all attributes of  $\beta$ :
     $\alpha \rightarrow \beta$  is preserved
  
```

Advantage: Polynomial time instead of exponential time to compute F^+ .

Slide 42: Example of Dependency Preservation Test

Key Concept: Practical Application

Given: $R=(A,B,C)$, $F=\{A\rightarrow B, B\rightarrow C\}$, $\text{Key}=\{A\}$

Decomposition: $R_1=(A,B)$, $R_2=(B,C)$

Test for preservation:

- $R_1 \cap R_2 = \{B\}$
- F_1 (from R_1) = $\{A\rightarrow B\}$
- F_2 (from R_2) = $\{B\rightarrow C\}$
- $(F_1 \cup F_2)^+$ contains $A\rightarrow B$, $B\rightarrow C$, and $A\rightarrow C$
- This equals F^+ ✓ Dependency preserved

Slide 43: Testing for BCNF

Key Concept: BCNF Violation Detection

To check if non-trivial FD $\alpha\rightarrow\beta$ violates BCNF:

1. Compute α^+ (attribute closure of α)
2. Check if α^+ includes ALL attributes of R
3. If not: α is not a superkey, so BCNF is violated

Shortcut: Suffices to check only FDs in F (not F^+) for R itself

- But for decomposed relations, MUST check restrictions of F^+

Example warning: When decomposing, subsets might violate BCNF even if original doesn't.

Slide 44: Testing Decomposition for BCNF

Key Concept: Checking Subrelations

For each relation R_i in decomposition:

- For every set $\alpha \subseteq R_i$, check if α^+ :
 - Either contains NO attributes from $R_i - \alpha$

- OR contains ALL attributes from R_i
- If violated: FD $\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i$ holds on R_i
- This FD violates BCNF on R_i

The point: Must be careful when checking decomposed relations - violations might emerge that weren't visible in the original schema.

Slide 45: BCNF Decomposition Algorithm

Key Concept: The Main Algorithm

Pseudocode:

```

result := {R}
done := false
compute  $F^+$ 

while not done:
    if there's a schema  $R_i$  in result NOT in BCNF:
        let  $\alpha \rightarrow \beta$  be a non-trivial FD such that:
            -  $\alpha \rightarrow R_i$  is not in  $F^+$ 
            -  $\alpha \cap \beta = \emptyset$ 
        result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha \cup \beta$ )
    else:
        done := true

```

Guarantees:

- Each R_i is in BCNF
- Decomposition is lossless-join

Slide 46: Example of BCNF Decomposition (Simple)

Key Concept: Step-by-Step Decomposition

Given: $R=(A,B,C)$, $F=\{A \rightarrow B, B \rightarrow C\}$, $\text{Key}=\{A\}$

Analysis:

- R NOT in BCNF ($B \rightarrow C$ but B is not superkey)

Decomposition:

- Choose FD $B \rightarrow C$ that violates BCNF
- $R_1 = (B, C)$ (the violating dependency)
- $R_2 = (A, B)$ (remaining attributes)

Result: Both R_1 and R_2 are in BCNF

Slide 47: Example of BCNF Decomposition (Complex)

Key Concept: Multi-Step Decomposition

Given: Complex banking relation

- $R = (\text{branch_name}, \text{branch_city}, \text{assets}, \text{customer_name}, \text{loan_number}, \text{amount})$
- $F = \{\text{branch_name} \rightarrow \text{assets} \text{ branch_city}, \text{loan_number} \rightarrow \text{amount} \text{ branch_name}\}$
- $\text{Key} = \{\text{loan_number}, \text{customer_name}\}$

Decomposition process:

- $R_1 = (\text{branch_name}, \text{branch_city}, \text{assets})$
- $R_2 = (\text{branch_name}, \text{customer_name}, \text{loan_number}, \text{amount})$
- $R_3 = (\text{branch_name}, \text{loan_number}, \text{amount})$
- $R_4 = (\text{customer_name}, \text{loan_number})$

Final result: $\{R_1, R_3, R_4\}$ all in BCNF

Slide 48: BCNF and Dependency Preservation - The Trade-off

Key Concept: When You Can't Have Both

Problem: Sometimes BCNF decomposition loses dependency preservation

Example:

- $R = (J, K, L)$
- $F = \{JK \rightarrow L, L \rightarrow K\}$
- Candidate keys: $\{JK, JL\}$

Issue: Any BCNF decomposition loses the ability to check $JK \rightarrow L$ without joining

- Therefore: Test for $JK \rightarrow L$ requires expensive joins
- Solution: Accept non-BCNF and use 3NF instead

Slide 49: Third Normal Form - Motivation

Key Concept: Why We Need 3NF

Problem with BCNF: Not always dependency preserving

Solution: 3NF provides:

1. Weaker constraint than BCNF (allows some redundancy)
2. Always dependency preserving
3. Always has lossless-join decomposition
4. **Checking FD violations doesn't require joins** (efficient)

Slide 50: 3NF Example

Key Concept: Example of 3NF

Given: $R=(J,K,L)$, $F=\{JK \rightarrow L, L \rightarrow K\}$

- Candidate keys: $\{JK, JL\}$

Is R in 3NF?

Check FD1: $JK \rightarrow L$

- JK is a superkey ✓ (second condition satisfied)

Check FD2: $L \rightarrow K$

- L is not a superkey
- But K is in candidate key {JK} ✓ (third condition satisfied)

Result: R IS in 3NF

Slide 51: Redundancy in 3NF

Key Concept: Trade-off - Some Redundancy Allowed

3NF allows redundancy in exchange for dependency preservation:

Example data:

J	L	K
j ₁	l ₁	k ₁
j ₂	l ₁	k ₁
j ₃	l ₁	k ₁
nul l	l ₂	k ₂

Problems:

- Relationship l₁, k₁ repeated three times
- Null value for J when representing l₂, k₂ relationship

Trade-off: Accept this redundancy for efficient dependency checking.

Slide 52: Testing for 3NF

Key Concept: Algorithm

Optimization: Need only check FDs in F, not F⁺

For each non-trivial FD $\alpha \rightarrow \beta$:

1. If α is a superkey: ✓ (2nd condition)
2. Else: Must check if each attribute in β is in some candidate key
 - This is the expensive part (requires finding all candidate keys)
 - NP-hard problem to test

Paradox: Testing 3NF is hard, but decomposing into 3NF is polynomial time!

Slide 53: 3NF Decomposition Algorithm

Key Concept: The Algorithm

Input: Canonical cover F_c

```
i := 0
for each FD  $\alpha \rightarrow \beta$  in  $F_c$ :
    if no schema  $R_j$  contains  $\alpha\beta$ :
        i := i + 1
         $R_i := \alpha\beta$ 

if no schema  $R_j$  contains a candidate key:
    i := i + 1
     $R_i :=$  any candidate key for R

return ( $R_1, R_2, \dots, R_i$ )
```

Properties:

- Each R_i in 3NF
- Decomposition is lossless-join AND dependency preserving

Slide 54: 3NF Decomposition (cont.) - Correctness

Key Concept: Why Algorithm Works

Guarantees:

- Each resulting relation schema is in 3NF
- Decomposition is dependency preserving (one relation for each FD in F_c)

- Decomposition is lossless-join (candidate key preserved)

Proof sketch:

- If R_i created from $\alpha \rightarrow \beta$, it's in 3NF by construction
- Candidate key in some R_j ensures lossless join
- Each FD in F_c has its own relation, preserving dependencies

Slide 55: 3NF Decomposition Example

Key Concept: Practical Application

Given: $\text{cust_banker_branch} = (\text{customer_id}, \text{employee_id}, \text{branch_name}, \text{type})$

FDs:

1. $\text{customer_id}, \text{employee_id} \rightarrow \text{branch_name}, \text{type}$
2. $\text{employee_id} \rightarrow \text{branch_name}$
3. $\text{customer_id}, \text{branch_name} \rightarrow \text{employee_id}$

Canonical cover F_c (branch_name extraneous in RHS of 1):

1. $\text{customer_id}, \text{employee_id} \rightarrow \text{type}$
2. $\text{employee_id} \rightarrow \text{branch_name}$
3. $\text{customer_id}, \text{branch_name} \rightarrow \text{employee_id}$

3NF Decomposition creates:

- $(\text{customer_id}, \text{employee_id}, \text{type})$
- $(\text{employee_id}, \text{branch_name})$
- $(\text{customer_id}, \text{branch_name}, \text{employee_id})$

Slide 56: 3NF Decomposition Example (Continued)

Key Concept: Optimization and Simplification

Observation: If some schemas are subsets of others, can be removed

- (employee_id, branch_name) is subset of (customer_id, branch_name, employee_id)
- Can be removed

Simplified 3NF:

- (customer_id, employee_id, type)
- (customer_id, branch_name, employee_id)

Note: Order of FD consideration might affect intermediate results, but simplified version is independent of order.

Slide 57: Comparison of BCNF and 3NF

Key Concept: Summary of Key Differences

Property	BCNF	3NF
Normal form strength	Stronger	Weaker
Dependency preservation	Not always	Always possible
Lossless decomposition	Always	Always
Redundancy	Minimal	Some allowed
Practical use	Better for design	Better for constraints

Key insight: 3NF always has a lossless-join, dependency-preserving decomposition. BCNF may not.

Slide 58: Design Goals

Key Concept: The Ultimate Objectives

Ideal design goals:

1. **BCNF** (strongest normal form)
2. **Lossless-join** (reconstruction possible)

3. **Dependency preservation** (efficient constraint checking)

In practice: Often must compromise:

- **Choose 1:** Accept non-BCNF (use 3NF instead)
- **Choose 2:** Accept redundancy but get dependency preservation

Note on SQL:

- SQL doesn't directly support FDs (except for keys)
- Can use assertions, but they're expensive to test
- Even with preserved dependencies, non-key FDs require expensive checking

Slide 59: Multivalued Dependencies (MVDs) - Introduction

Key Concept: Beyond Functional Dependencies

Motivation: Some relations in BCNF still have anomalies (like the classes example)

MVD Definition: $\alpha \twoheadrightarrow \beta$ holds on R if:

- For any two tuples t_1 and t_2 with same α values
- There exist tuples t_3 and t_4 such that:
 - All four have same α values
 - t_3 has t_1 's β values and t_2 's other values
 - t_4 has t_2 's β values and t_1 's other values

Intuition: Values of β are independent of values in R - α - β for each α

Slide 60: MVD Tabular Representation

Key Concept: Visual Understanding

Table showing $\alpha \twoheadrightarrow \beta$:

	α	β	R - α - β
--	----------	---------	------------------------

t 1	a ₁ . . . a _i	a _{i+1} . . . a _j	a _{j+1} . . . a _n
t 2	a ₁ . . . a _i	b _{i+1} . . . b _j	b _{j+1} . . . b _n
t 3	a ₁ . . . a _i	a _{i+1} . . . a _j	b _{j+1} . . . b _n
t 4	a ₁ . . . a _i	b _{i+1} . . . b _j	a _{j+1} . . . a _n

Key pattern: If you swap the non- α attributes between two tuples with same α , you get two more valid tuples.

Slide 61: MVD Example

Key Concept: Classes Example Revisited

Relations Y, Z, W (partitioning all attributes):

- $Y \twoheadrightarrow Z$ means: Given Y value, there's a set of Z values independent of W

For classes table: course $\twoheadrightarrow$ teacher and course $\twoheadrightarrow$ book

- Given a course, there's a fixed set of teachers (independent of book)
- Given a course, there's a fixed set of books (independent of teacher)
- MVD captures this independence

Relationship:

- If $Y \rightarrow Z$ then $Y \twoheadrightarrow Z$ (every FD is also MVD)
- But MVD is more general than FD

Slide 62: Use of Multivalued Dependencies

Key Concept: Two Applications

Uses of MVDs:

1. **Test relations:** Check if instance satisfies given MVDs
2. **Specify constraints:** Describe valid relations on schema

Important property:

- Can construct relations satisfying MVDs by adding tuples
- But can't always have minimal relations

When to use:

- When relations have independent repeating groups
- When normalization to BCNF leaves anomalies

Slide 63: Theory of MVDs

Key Concept: Closure and Axioms

Key rules:

1. If $\alpha \rightarrow \beta$ then $\alpha \twoheadrightarrow \beta$ (every FD is MVD)

Closure D^+ :

- All FDs and MVDs logically implied by D
- Complex to compute for MVDs (use inference rules)
- Much more involved than FD closure

For practice: Most MVD cases are simple and don't require full closure computation.

Slide 64: Fourth Normal Form (4NF)

Key Concept: Handling Multivalued Dependencies

Definition: R is in 4NF with respect to D (FDs + MVDs) if:

- For all non-trivial MVDs $\alpha \twoheadrightarrow \beta$ in D^+ :
 - Either $\alpha \twoheadrightarrow \beta$ is trivial ($\beta \subseteq \alpha$ or $\alpha \cup \beta = R$)
 - Or α is a superkey

Key insight:

- 4NF subsumes BCNF (stricter condition)
- If R is in 4NF, it's automatically in BCNF
- Eliminates anomalies from independent repeating groups

Slide 65: Restriction of MVDs**Key Concept: Handling MVDs in Decomposition**

When decomposing, must track MVDs affecting subrelations

Restriction of D to Ri:

- All FDs in D^+ with only R_i attributes
- All MVDs $\alpha \twoheadrightarrow (\beta \cap R_i)$ where $\alpha \subseteq R_i$ and $\alpha \twoheadrightarrow \beta$ in D^+

Example: If $\text{course} \twoheadrightarrow \text{teacher}$ in original, and decompose into separate tables, must track this constraint properly.

Slide 66: 4NF Decomposition Algorithm**Key Concept: The Algorithm**

Algorithm (similar to BCNF but with MVDs):

```
result := {R}
done := false
compute  $D^+$ 
Let  $D_i$  = restriction of  $D^+$  to  $R_i$ 

while not done:
    if schema  $R_i$  NOT in 4NF:
        let  $\alpha \twoheadrightarrow \beta$  be non-trivial MVD such that
             $\alpha \rightarrow R_i$  not in  $D_i$  and  $\alpha \cap \beta = \emptyset$ 
        result := (result -  $R_i$ )  $\cup$  ( $R_i$  -  $\beta$ )  $\cup$  ( $\alpha \cup \beta$ )
    else:
        done := true
```

Guarantees: Each R_i in 4NF, lossless-join decomposition

Slide 67: 4NF Example

Key Concept: Practical 4NF Decomposition

Given: $R=(A,B,C,G,H,I)$, $D=\{A \twoheadrightarrow B, B \twoheadrightarrow HI, CG \twoheadrightarrow H\}$

Not in 4NF ($A \twoheadrightarrow B$ and A is not superkey)

Decomposition process:

1. $R_1 = (A, B)$ - from $A \twoheadrightarrow B$
2. $R_2 = (A, C, G, H, I)$ - remainder (not 4NF)
3. Since $B \twoheadrightarrow HI$ and recursively: $R_3=(C,G,H)$, $R_4=(A,C,G,I)$
4. Continue: $R_5=(A,I)$, $R_6=(A,C,G)$

Final 4NF decomposition: $\{R_1, R_3, R_5, R_6\}$

Slide 68: Further Normal Forms

Key Concept: Beyond 4NF

More advanced concepts:

1. **Join Dependencies:** Generalize MVDs
 - Lead to **Project-Join Normal Form (PJNF)** or Fifth Normal Form
2. **Domain-Key Normal Form:**
 - Even more general constraints
 - Hard to reason with

Practical note:

- Rarely used (hard to work with)
- No complete inference rule systems
- BCNF or 3NF suffice for most real-world designs

Slide 69: Overall Database Design Process

Key Concept: Where Normalization Fits

How schema R is obtained:

1. **From E-R diagram:** Converting entities and relationships
2. **From universal relation:** Single relation with all attributes
3. **From ad hoc design:** Pragmatic collection of attributes

Normalization process:

- Takes R and breaks into smaller relations
- Tests and converts to proper normal form
- Eliminates anomalies

Lesson: Normalization is post-processing after initial schema design.

Slide 70: ER Model and Normalization Relationship

Key Concept: Good E-R Design Reduces Normalization

Best case: Carefully designed E-R diagram

- Proper entity identification
- Correct attributes assignment
- Results in tables needing minimal normalization

Real-world E-R imperfections:

- Non-key attributes can have FDs to other attributes
- Example: employee entity with department_number → department_address
- Indicates poor design (should be separate entity)

Relationship MVDs:

- Possible but rare (mostly binary relationships)

- Less common than entity-level issues

Slide 71: Denormalization for Performance

Key Concept: When to Break Normalization Rules

Scenario: Joining causes performance issues

- Example: Display customer_name with account balance requires joining account + depositor

Alternative 1: Denormalized relation

- Combine account + depositor attributes
- Pros: Faster lookup
- Cons: Extra space, update overhead, coding complexity, error-prone

Alternative 2: Materialized View (Better!)

- Define as account $\bowtie$ depositor
- Pros: Same speed as denormalization
- Cons: Same space/update overhead, but no extra coding

Decision: Use denormalization only when performance truly requires it.

Slide 72: Other Design Issues

Key Concept: Problems Not Caught by Normalization

Example of bad design (not caught by normal forms):

✗ WRONG - Time-Series as Tables:

```
earnings_2004(company_id, earnings)
earnings_2005(company_id, earnings)
earnings_2006(company_id, earnings)
```

- All in BCNF, but querying across years is impossible
- Need new table each year

✗ WRONG - Time-Series as Crosstab:

```
company_year(company_id, earnings_2004, earnings_2005, earnings_2006)
```

- Also BCNF, also bad (querying is difficult)
- Need new column each year

✓ CORRECT:

```
earnings(company_id, year, amount)
```

- Easy to query any year range
- Natural temporal representation

Slide 73: Modeling Temporal Data - Introduction

Key Concept: Time-Varying Information

Definition: Temporal data have **valid time intervals**

- When data are valid (not always current)

Related concepts:

- **Snapshot:** Data value at particular point in time
- **Valid time:** Time interval during which data are valid

ER model extensions (no standard yet):

- Temporal attributes (address at different times)
- Temporal entities (when account exists)
- Temporal relationships (when customer owned account)

Challenge: FDs change with time (e.g., $\text{customer_id} \rightarrow \text{address}$ no longer holds if address changes)

Slide 74: Temporal FDs and Data Modeling

Key Concept: Time-Aware Constraints

Problem: FDs like $\text{customer_id} \rightarrow \text{street, city}$ don't hold with temporal data

- Same customer has different addresses over time

Solution: Temporal Functional Dependency

- $X \rightarrow_t Y$ means: FD $X \rightarrow Y$ holds on all snapshots
- For all time points, X determines Y

Practical approach: Add time attributes to relations

- Example: $\text{course}(\text{course_id}, \text{title}) \rightarrow \text{course}(\text{course_id}, \text{title}, \text{start}, \text{end})$
- Constraint: No overlapping valid times for same course
- Challenge: Hard to enforce efficiently

Foreign key complexity: Should transcript reference current course info or course info at enrollment time?

Slide 75: End of Chapter Marker

This slide marks the end of the main chapter content.

Slide 76-77: Proof of Correctness of 3NF Decomposition Algorithm

These final slides provide the mathematical proof that the 3NF decomposition algorithm is correct:

Key proofs:

1. **Dependency preservation:** Since one relation exists for each FD in canonical cover
2. **Lossless join:** Candidate key is in one relation; closure guarantees reconstruction
3. **3NF satisfaction:** Each resulting relation satisfies 3NF conditions by construction

The proof covers two cases for attributes in β (either extraneous removal or candidate key membership).